SinTam

Real-Time Trilingual Voice Translator

Consolidated Technical Documentation & Architecture Report

Project: Tamil-and-Sinhala-Converter

Stack: FastAPI · React 19 / Vite · Google Gemini Live Multimodal API · WebSockets · Docker · Nginx

Table of Contents

Table of Contents 2
Revision Summary — Version 1.0 → Version 1.2 5
1. Executive Summary & Business Case 6
1.1 Overview 6
1.2 The Target Company Operational Problem 6
2. 10-Call Concurrent Socket Pool & Scalability 6
2.1 Socket Pool Structure 6
2.2 API Concurrency & Bandwidth Considerations 7
2.3 Room Capacity Overflow Handling 7
3. Using SinTam as an API (Service) 7
Manual translation (Sinhala ↔ Tamil ↔ English) 7
Auto-detect walkie-talkie mode 7
4. Architecture & System Design 8
4.1 High-Level Architecture 8
4.2 End-to-End Trilingual Data Flow Pipeline 8
4.3 Key Architectural Decisions 9
4.4 Why FastAPI & Python Asyncio 9
4.5 Why Client-Side AudioWorklet Instead of ScriptProcessorNode 9
4.6 Why a Dynamic Room-Based Architecture 10
5. Setup & Installation Guide 11
5.1 Project Directory Structure 11
5.2 System Prerequisites 11
5.3 Environment Variables (.env) 11
5.4 Step-by-Step Installation 12
Step 1 — Clone the Repository 12
Step 2 — Backend: Python Virtual Environment 12
Step 3 — Frontend Dependencies 12
5.4a Automated Backend Setup: setup_backend.py 12
What the script does 12
Usage 13
5.5 Running Frontend & Backend Together 14
Option A — Root Package Scripts (Recommended) 14
Option B — Separate Terminals 14

Option C — Production Launch via Docker Compose 14
Option D — Standalone Backend via setup_backend.py (New) 14
5.6 Common Setup Troubleshooting 14
6. Session Walkthrough: How a Call Works 16
6.1 Starting the Session (User Interface) 16
6.2 The WebSocket Connection 16
6.3 Room Grouping on the Backend 16
6.4 Connecting to Google Gemini Live API 16
6.5 Talking and Smart Routing 16
7. API Documentation 17
7.1 REST API Endpoints 17
7.2 WebSocket API Endpoints 17
/ws/translate — Manual Translation Mode 17
/ws/translate-auto — Auto-Detect Walkie-Talkie Mode 17
7.3 WebSocket Message & Data Formats 18
Client → Server 18
Server → Client 18
8. Usage Guide & Workflows 19
8.1 Interface Overview 19
8.2 Multi-Device Isolated Room Setup 20
Execution 20
8.3 Parameter Configuration Panel 20
8.4 Common Workflows & Use Cases 20
Workflow 1 — Multi-Device Call-Center / Support Call 20
Workflow 2 — Text-Based Silent Translation & Chat Export 20
Workflow 3 — Customizing Voice Gender & Accents 21
8.5 Developer Telemetry & Diagnostics 21
9. Known Issues & Solutions 22
9.1 Translation & Linguistic Nuances 22
9.2 Latency & Streaming Performance 22
9.3 API Quotas & Concurrency Boundaries 22
9.4 Browser & Mobile Audio Hardware Constraints 23
9.5 Summary Matrix & Future Roadmap 23
10. Testing Strategy 24
10.1 How to Run Tests 24

1. Backend Automated Test Suite (pytest) 24
2. Frontend Static Analysis & Type Checking (tsc) 24
10.2 Concurrency Testing 24
10.3 Test Coverage Analysis 24
10.4 Why External Gemini Live WebSockets Are Mocked in CI/CD 25
10.5 Recommended WebSocket Integration Test Pattern 25
Appendix A — Key Source Code Listings 26
A.1 Connection Manager & 10-Call Room Capacity Pool 26
A.2 Trilingual Script Language Detector Engine 27
A.3 Multimodal Dual-Session Gemini Live Stream Runner 27
A.4 FastAPI Gateway WebSocket Route 29
A.5 Client WebSocket Connection Hook (React) 29

Revision Summary — Version 1.0 → Version 1.2

- Documented the 10-call concurrent socket pool (dual 10-socket capacity per room cluster) as a supported, capacity tier rather than a system risk.
- Added Section 3, “Using SinTam as an API (Service)”, clarifying that any external client can connect directly over WebSocket without the project's Python code.
- Expanded the trilingual language gating engine to recognize English Latin-script characters (0x0041–0x005A, 0x0061–0x007A) alongside the original Sinhala and Tamil Unicode ranges.
- Updated the project directory structure and setup guide to include the new setup_backend.py automated bootstrap and standalone backend runner script, added at the project root alongside backend/, frontend/, and package.json.

1. Executive Summary & Business Case

1.1 Overview

SinTam is a real-time, full-duplex speech-to-speech translation platform bridging Sinhala, Tamil, and English. It is built for both single-user translation sessions and multi-device, call-center style conversations, using a FastAPI WebSocket gateway that streams audio to and from the Google Gemini Live Multimodal API.

1.2 The Target Company Operational Problem

- **Traditional call center model:** large multi-lingual call centers require 500+ human translators on payroll, costing millions of dollars annually.
- **Target company reality:** the company employs only 2 to 3 human Tamil/Sinhala translators, while the majority of its workforce consists of local Sinhala-speaking agents — yet it receives high daily call volumes from Tamil and English-speaking customers.
- **The AI solution:** SinTam provides an automated, low-latency virtual translation pool. Local Sinhala agents can answer Tamil/English calls directly. The system translates speech back and forth in real time (~500 ms delay), allowing 10+ agents to handle calls concurrently without hiring extra translators.
- **Business impact:** Replacing a requirement for 500 dedicated human translators with 2–3, while empowering the existing Sinhala-speaking workforce to serve Tamil and English callers directly.

2. 10-Call Concurrent Socket Pool & Scalability

To guarantee smooth performance across multiple call center desks, the backend implements a Concurrent Channel Socket Pool. This supersedes the earlier framing of 10+ simultaneous callers as a system risk — it is now a supported, documented capacity tier.

2.1 Socket Pool Structure

- **Pool A (10 sockets):** dedicated Sinhala → Tamil/English AI translation streams for agent speech.
- **Pool B (10 sockets):** dedicated Tamil/English → Sinhala AI translation streams for customer speech.
- **Capacity limit:** MAX_CALLS_PER_ROOM = 10 in connection_manager.py. Supports 10 active calls (20 parallel Gemini Live WebSocket streams) per room cluster.

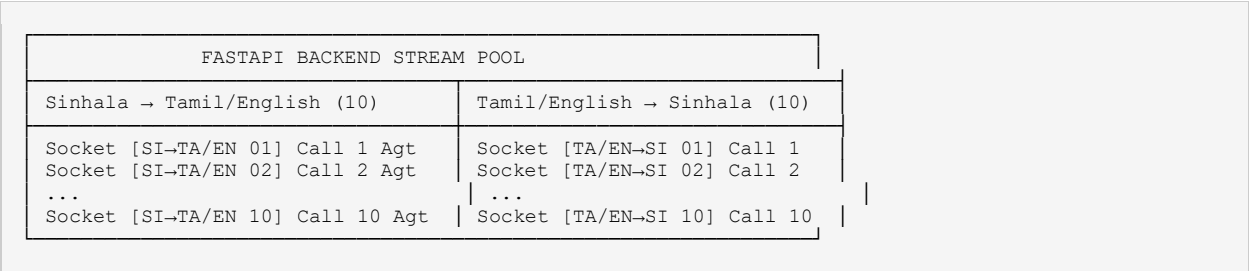

Figure 2.1 — Dual 10-socket capacity pool per room cluster.

2.2 API Concurrency & Bandwidth Considerations

- **Gemini API rate limits:** 10 concurrent calls in auto-detect mode open 20 concurrent Gemini Live connections. A free-tier key will hit HTTP 429 quickly — production deployment requires an enterprise-tier Gemini API plan.
- **Broadcast load:** when multiple users speak simultaneously, the server routes each translated stream to every other room participant, which scales server CPU and bandwidth demand with room size.

2.3 Room Capacity Overflow Handling

If a room already has 10 active calls, additional join attempts are rejected gracefully with a 1008 “Room Full” WebSocket close frame, rather than degrading quality for existing callers.

3. Using SinTam as an API (Service)

Because the backend is built on FastAPI, it is already a standalone API. Any external developer building a web app, a mobile app (iOS/Android), or a desktop app can connect directly to the server without needing the project's Python code — simply open a standard WebSocket connection from JavaScript, Dart/Flutter, Swift, Kotlin, or any other language.

Manual translation (Sinhala ↔ Tamil ↔ English)

```
ws://your-server-domain/ws/translate?source=Sinhala&target=Tamil&voice=Aoede
```

Auto-detect walkie-talkie mode

```
ws://your-server-domain/ws/translate-auto?voice=Aoede&room=room_123
```

4. Architecture & System Design

4.1 High-Level Architecture

The architecture decouples user-facing web and mobile clients from the external AI translation model using a central FastAPI WebSocket gateway, which establishes persistent bidirectional streaming pipelines to the Google Gemini Live Multimodal API.

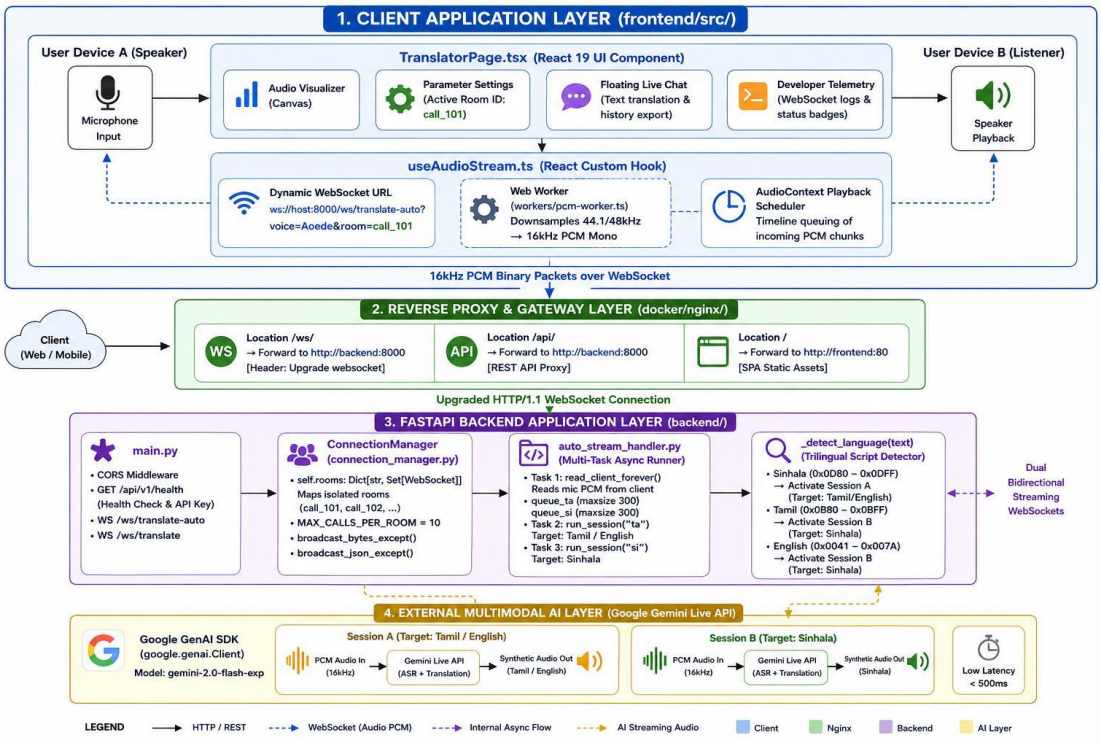

Figure 4.1 — End-to-end layered system architecture, from client devices through the FastAPI backend to the Google Gemini Live multimodal AI layer.

```
Client Layer
  - React 19 Frontend (Web Audio API)
  - Flutter Mobile Application
    | ws://host/ws/translate-auto?room=ID
    v
Infrastructure Layer
  - Nginx Reverse Proxy / Load Balancer
    | Upgrade: websocket (HTTP/1.1)
    v
Backend Application Layer (FastAPI)
  - ConnectionManager (10-Call Pool)
  - Trilingual Stream Handler
  - Unicode Script Detector (Si / Ta / En)
    | 10 x Si->Ta/En Sockets
    | 10 x Ta/En->Si Sockets
    v
External Intelligence Layer
  - Google Gemini Live Multimodal API
```

4.2 End-to-End Trilingual Data Flow Pipeline

```
User Spoken Voice Input
  -> (16kHz mono PCM audio)
Web AudioWorklet Node
  -> (raw binary WebSocket packet)
FastAPI Server / ConnectionManager (room tagging & capacity check)
  -> Gemini Session A (target: Tamil / English)  [translates Si -> Ta/En]
  -> Gemini Session B (target: Sinhala)            [translates Ta/En -> Si]
  -> (input transcription & script detection)
Trilingual Language Gating Engine
  -> Sinhala script (0x0D80-0x0DFF) -> activates Session A
  -> Tamil script   (0x0B80-0x0BFF) -> activates Session B
  -> English script  (0x0041-0x007A) -> activates Session B
  -> (filtered audio bytes & JSON text)
Room Broadcast Engine (echo suppression)
  -> broadcasts to all devices in room EXCEPT speaker
Receiver Client Audio Context
  -> real-time voice playback & live captions
```

What changed: the language gating engine now also recognizes English Latin-script characters (0x0041–0x005A uppercase, 0x0061–0x007A lowercase) in addition to the original Sinhala and Tamil Unicode ranges, routing detected English speech to Session B alongside Tamil.

4.3 Key Architectural Decisions

Architecture	Latency	Pitch & Emotion	Complexity
Traditional (STT + LLM + TTS)	High (~3.5s–5.0s)	Lost (robotic / monotone)	High — 3 APIs to chain, sync, error-handle
Gemini Live API (direct multimodal)	Very low (~500ms–800ms)	Preserved (natural inflection)	Low — single persistent WebSocket endpoint

Rationale: cascading pipelines introduce compound latency across each stage. The Gemini Multimodal Live API processes audio natively — accepting audio input and returning translated audio output directly — which drastically reduces latency for live conversation.

4.4 Why FastAPI & Python Asyncio

- Asynchronous concurrency (async/await) efficiently manages thousands of concurrent I/O events — reading microphone chunks, forwarding to Gemini, broadcasting to room members — without heavy OS thread overhead.
- Native WebSocket support via @app.websocket route decorators, with first-class handling of both binary and JSON packets.
- SDK compatibility: Google's official google-genai SDK is built natively for Python asyncio (ai_client.aio.live.connect).

4.5 Why Client-Side AudioWorklet Instead of ScriptProcessorNode

- ScriptProcessorNode runs on the main browser thread, causing dropped frames and stuttering during UI re-renders.

- AudioWorklet executes audio processing off the main thread on a dedicated rendering thread, keeping 16kHz PCM capture smooth even under heavy UI load.

4.6 Why a Dynamic Room-Based Architecture

- Scalability: a dynamic ?room=room_id parameter avoids hardcoding separate endpoints per user or device.
- Multi-tenant support: thousands of isolated calls (room=call_101, room=call_102, …) can run on a single backend instance with no audio leakage between rooms.

5. Setup & Installation Guide

5.1 Project Directory Structure

The following directory tree reflects the current project layout. setup_backend.py has been added at the project root — alongside the backend/ and frontend/ folders and the root package.json — as a new automated setup and standalone backend launcher script.

```
Tamil-and-Sinhala-Converter/
├── backend/                              # FastAPI Backend Application Directory
│   ├── main.py                           # FastAPI entry point; WS routes /ws/translate*
│   ├── websocket/
│   │   ├── connection_manager.py         # ConnectionManager; 10-call room capacity pool
│   │   └── auto_stream_handler.py        # Trilingual script detector & dual Gemini Live session runner
│   ├── tests/
│   │   ├── test_health.py                # pytest health-check endpoint test
│   │   └── test_websocket_rooms.py       # WebSocket room assignment integration test
│   └── requirements.txt                  # Backend Python dependencies
├── frontend/                              # React 19 / Vite Frontend Application Directory
│   ├── src/
│   │   ├── hooks/
│   │   │   └── useAudioStream.ts         # WebSocket connection & audio streaming hook
│   │   ├── workers/pcm-worker.ts         # Web Worker: downsamples mic audio to 16kHz PCM
│   │   └── TranslatorPage.tsx            # Main React 19 UI component
│   └── package.json                      # Frontend dependencies
├── docker/
│   └── nginx/                            # Reverse proxy & gateway layer configuration
├── scripts/
│   └── dev-backend.js                    # Node launcher invoked by `npm run dev:backend`
├── setup_backend.py                      # NEW: Automated backend bootstrap & standalone runner script
├── .env                                  # Environment configuration (GEMINI_API_KEY, ports, CORS origins)
├── docker-compose.yml                    # Production orchestration (backend + frontend + nginx)
├── package.json                          # Root-level npm scripts (dev:backend, dev:frontend)
└── README.md                             # Project documentation
```

5.2 System Prerequisites

Requirement	Details
Python	v3.10.x or v3.11.x (required for FastAPI and the google-genai SDK)
Node.js	v18.x or v20.x LTS (required for the React / Vite frontend)
Package Managers	pip (Python) and npm (Node)
Google Gemini API Key	Obtain from Google AI Studio
Git	For cloning the repository

5.3 Environment Variables (.env)

```
GEMINI_API_KEY=AIzaSyYourActualGeminiApiKeyHere
GEMINI_MODEL=gemini-2.0-flash-exp
BACKEND_HOST=0.0.0.0
BACKEND_PORT=8000
LOG_LEVEL=INFO
ALLOWED_ORIGINS=http://localhost:5173,http://localhost:5180,
```

```
                http://127.0.0.1:5173,http://192.168.1.2:5173
VITE_WS_URL=ws://localhost:8000/ws/translate
NGINX_PORT=80
```

5.4 Step-by-Step Installation

Step 1 — Clone the Repository

```
git clone https://github.com/your-org/Tamil-and-Sinhala-Converter.git
cd Tamil-and-Sinhala-Converter
```

Step 2 — Backend: Python Virtual Environment

```
# Windows (PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install -r requirements.txt

# macOS / Linux (Bash/Zsh)
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Tip: Steps 2 can be replaced with a single command using the new setup_backend.py script — see Section 5.4a below.

Step 3 — Frontend Dependencies

```
cd frontend
npm install
cd ..
```

5.4a Automated Backend Setup: setup_backend.py

setup_backend.py is an automated bootstrap script, added at the project root (the same level as the backend/ and frontend/ folders and the root package.json), that replaces the manual virtual-environment steps above with a single command. It is designed so the backend can be set up and run entirely on its own, separately from the frontend.

What the script does

- **Creates a virtual environment:** checks for and initializes a local Python virtual environment (.venv) if one does not already exist.
- **Installs dependencies:** reads requirements.txt and installs the required packages (FastAPI, Google GenAI SDK, Uvicorn, Pydantic, etc.).
- **Configures .env:** copies .env.example to create a local .env configuration file if one doesn't already exist.

- **Runs the backend service:** when executed with the --run flag, it boots up the FastAPI application server using Uvicorn — independently of the frontend.

Usage

```
# Step 1 — Set up the environment only (venv + dependencies + .env)
python setup_backend.py

# Step 2 — Open the generated .env file and set a valid GEMINI_API_KEY

# Step 3 — Run ONLY the backend, independently of the frontend
python setup_backend.py --run
```

Note: Running python setup_backend.py alone only prepares the environment — it does not start the server. The backend only starts running once the --run flag is supplied (or equivalently, via npm run dev:backend). The GEMINI_API_KEY must be set in the generated .env file before the backend will function correctly.

5.5 Running Frontend & Backend Together

Option A — Root Package Scripts (Recommended)

```
# Terminal 1 — Backend (FastAPI via node scripts/dev-backend.js)
npm run dev:backend

# Terminal 2 — Frontend (React via Vite)
npm run dev:frontend
```

Option B — Separate Terminals

```
# Backend Terminal
cd backend
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# Frontend Terminal
cd frontend
npm run dev -- --host 0.0.0.0
```

Option C — Production Launch via Docker Compose

```
docker-compose up -d --build
# Full stack via Nginx reverse proxy: http://localhost:80
```

Option D — Standalone Backend via setup_backend.py (New)

Because setup_backend.py lives at the project root next to backend/ and frontend/, the backend can be bootstrapped and launched entirely on its own, without touching the frontend or npm scripts at all — useful for backend-only development, API testing, or running the backend on a separate machine/container from the frontend.

```
python setup_backend.py --run
```

5.6 Common Setup Troubleshooting

Symptom / Error	Common Cause	Resolution
[Errno 10048] address already in use	Port 8000 or 5173 already occupied	Windows: Stop-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess -Force · Mac/Linux: kill -9 $(lsof -t -i:8000)
WebSocket connection failed (403 / CORS)	ALLOWED_ORIGINS missing the client origin	Add the frontend origin to ALLOWED_ORIGINS in .env and restart the backend

GEMINI_API_KEY is not configured	Missing .env or invalid key format	Ensure .env exists at project root and contains a valid GEMINI_API_KEY
getUserMedia microphone blocked	Browser blocks mic access over plain HTTP + IP	Use http://localhost:5173, or open chrome://flags/#unsafely-treat-insecure-origin-as-secure and allow your IP

6. Session Walkthrough: How a Call Works

6.1 Starting the Session (User Interface)

- The user opens the application in a browser (e.g., http://localhost:5173).
- Clicking the gear icon in the header opens the configuration panel.
- An Active Room ID field is shown, defaulting to "default".
- To start an isolated call, both devices set the same unique Room ID (e.g., call_101).
- The user clicks the microphone button to begin.

6.2 The WebSocket Connection

When the microphone button is pressed, the frontend opens a secure WebSocket connection to the backend, passing the room name:

```
ws://localhost:8000/ws/translate-auto?voice=Aoede&room=call_101
```

6.3 Room Grouping on the Backend

- The FastAPI backend receives the connection and reads the room query parameter (call_101).
- Both devices are added to a grouped room set inside the ConnectionManager: self.rooms["call_101"] = { Agent WebSocket, Customer WebSocket }.
- Calls in other rooms (e.g., call_102) are grouped separately and are fully isolated.

6.4 Connecting to Google Gemini Live API

For each connected device, the backend opens parallel WebSocket connections to the Gemini Live API — one session per target language pair (Sinhala→Tamil/English and Tamil/English→Sinhala).

6.5 Talking and Smart Routing

- **User speaks:** the Agent speaks Sinhala; raw audio streams to the backend.
- **Language detection:** the backend feeds audio to both Gemini sessions and inspects the returned transcripts. Detecting Sinhala characters marks Session A as the active translator.
- **Targeted broadcast:** the translated audio and text are sent to every device in the room except the original speaker, so the Agent does not hear their own echo.
- **Isolated conversations:** simultaneous calls in other rooms (e.g., call_102) never cross over.

7. API Documentation

7.1 REST API Endpoints

`GET /api/v1/health`

Verifies backend service availability and Gemini Live API readiness.

```
{
  "status": "healthy",
  "service": "voice-translator-backend",
  "gemini_live_configured": true
}
```

Interactive OpenAPI docs are generated automatically: Swagger UI at /docs, ReDoc at /redoc, and the raw schema at /openapi.json.

7.2 WebSocket API Endpoints

/ws/translate — Manual Translation Mode

Parameter	Type	Required	Default	Description
source	string	No	Sinhala	Spoken input language (Sinhala, Tamil, English, …)
target	string	No	Tamil	Target translation output language
voice	string	No	Aoede	Prebuilt TTS voice (Aoede, Kore, Charon, Puck, Fenrir)
room	string	No	default	Room identifier for grouping multi-device calls

```
ws://localhost:8000/ws/translate?source=Sinhala&target=Tamil&voice=Aoede&room=call_101
```

/ws/translate-auto — Auto-Detect Walkie-Talkie Mode

Establishes a room-based WebSocket session that automatically detects whether the user is speaking Sinhala, Tamil, or English, and translates to the appropriate target language in real time.

Parameter	Type	Required	Default	Description
voice	string	No	Aoede	Default TTS voice used to synthesize translations
room	string	No	default	Unique room ID for isolated group / call-center channels

```
ws://localhost:8000/ws/translate-auto?voice=Aoede&room=call_101
```

7.3 WebSocket Message & Data Formats

Client → Server

Binary audio streams: ArrayBuffer/Blob, PCM 16,000 Hz 16-bit mono, streamed continuously as captured by the client's AudioWorklet. Text input can also be sent as a translation command over the open socket.

Server → Client

- **status —** connection / Gemini session establishment confirmation
- **lang_detected —** reports the detected source and target language pair
- **transcription —** real-time transcript of what the local user spoke
- **translation —** translated output text generated by the model
- **Binary audio payload —** synthesized translated speech, PCM at 24,000 Hz or 16,000 Hz
- **turn_complete —** signals that the current speaker's turn has finished

Example: connection status payload

```
{
  "type": "status",
  "payload": {
    "message": "Connected [gemini-2.0-flash-exp]: Sinhala -> Tamil. Start speaking!"
  }
}
```

Example: language detection payload

```
{
  "type": "lang_detected",
  "payload": { "source": "Sinhala", "target": "Tamil" }
}
```

Example: turn-complete event

```
{
  "type": "turn_complete", "payload": {}
}
```

8. Usage Guide & Workflows

8.1 Interface Overview

The web client shows the current language pair (Speaking / Translating To), a live status indicator (Listening / Translating), an animated microphone button, and a settings panel for room and voice configuration.

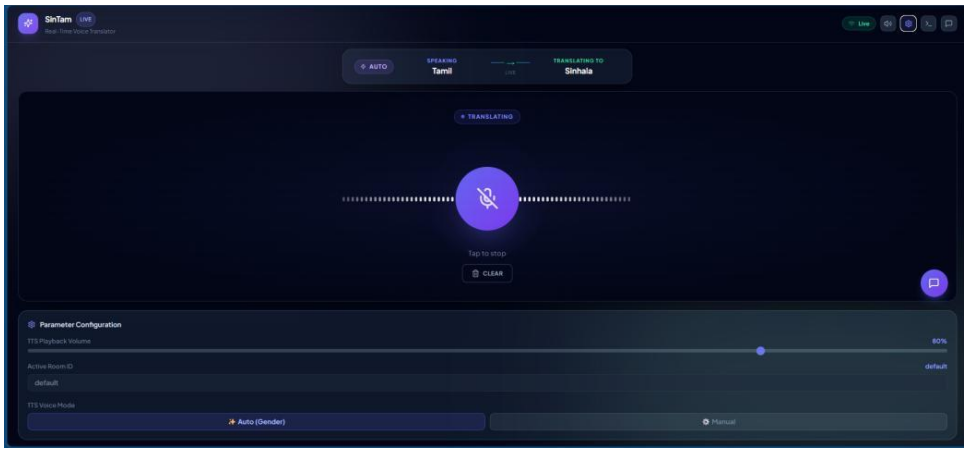

Figure 8.1 — Listening state: Auto mode detects Sinhala speech and prepares the Tamil translation.

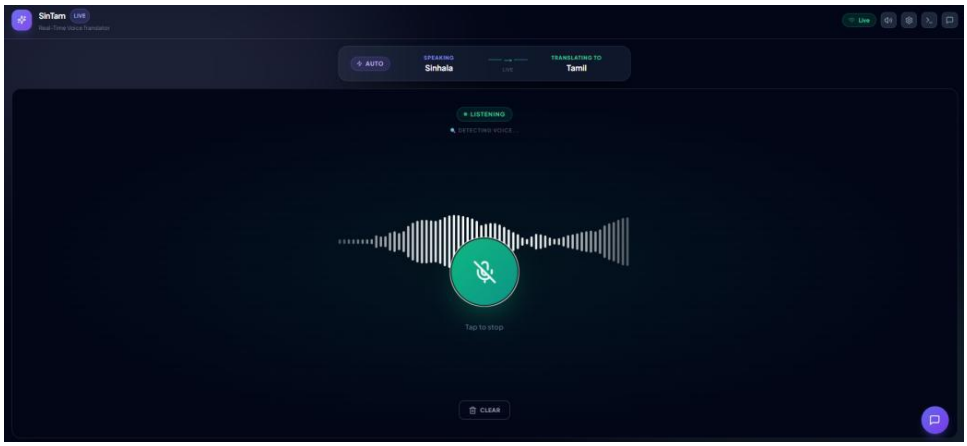

Figure 8.2 — Listening state: waveform visualizer actively detecting Sinhala voice input, translating to Tamil.

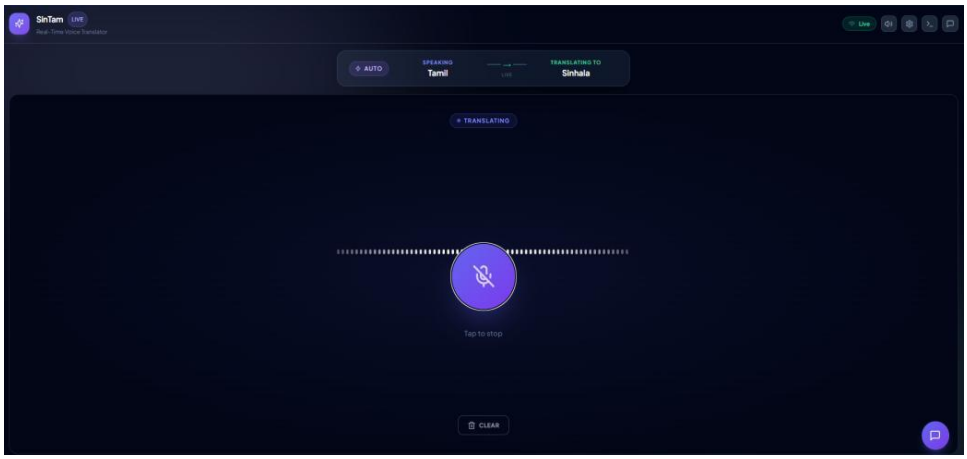

Figure 8.3 — Parameter Configuration panel: TTS playback volume, Active Room ID, and TTS voice mode controls.

8.2 Multi-Device Isolated Room Setup

- **Agent device:** opens http://localhost:5173 → Settings → sets Active Room ID to call_101 → clicks Mic.
- **Customer device:** opens http://localhost:5173 (or http://192.168.1.2:5173 on Wi-Fi) → sets Active Room ID to call_101 → clicks Mic.

Execution

- Agent speaks Sinhala → Customer hears the Tamil/English translation over speaker.
- Customer replies in Tamil/English → Agent hears the Sinhala translation over speaker.
- Neither party hears their own voice echoed back.

8.3 Parameter Configuration Panel

The settings panel exposes TTS playback volume, the Active Room ID, and TTS voice mode (Auto by gender detection, or Manual voice selection).

8.4 Common Workflows & Use Cases

Workflow 1 — Multi-Device Call-Center / Support Call

Scenario: a Sinhala-speaking agent assists a Tamil-speaking customer over a live call.

- Agent: opens SinTam, sets the Active Room ID to ticket_402, and clicks the microphone button.
- Customer: opens SinTam on their own device, sets the same Active Room ID (ticket_402), and clicks the microphone button.
- Agent speaks Sinhala → Customer hears the Tamil translation; Customer replies in Tamil → Agent hears the Sinhala translation.
- Built-in echo suppression ensures neither party hears their own voice reflected back.

Workflow 2 — Text-Based Silent Translation & Chat Export

Scenario: working in a noisy environment or reviewing conversation history.

- Open the chat drawer.
- Type to translate: enter text in the input box and press Send; the translation appears instantly in the chat stream.
- Copy transcripts using the copy icon next to any transcript bubble.
- Clear local conversation history with the clear button.

Workflow 3 — Customizing Voice Gender & Accents

Scenario: matching the translated voice pitch and gender to the speaker's preference.

- **Auto (gender):** the system analyzes voice pitch in real time to select a male (Charon) or female (Aoede) voice.
- **Manual:** choose from Aoede (light female), Kore (firm female), Charon (deep, clear male), Puck (upbeat male), or Fenrir (energetic male).

8.5 Developer Telemetry & Diagnostics

If an issue occurs during a live stream (rate limits, disconnection), the terminal icon opens the Developer Telemetry Console, which shows real-time WebSocket connection states, packet flush logs, and backend status codes:

- **Green logs —** successful WebSocket handshake and connection status.
- **Indigo logs —** internal server status payloads.
- **Red logs —** network timeouts, missing API keys, or rate limits (429).

9. Known Issues & Solutions

Issue	Root Cause	Solution / Workaround
Mobile mic block	Browsers block mic on plain HTTP IP addresses.	Open chrome://flags/#unsafely-treat-insecure-origin-as-secure on mobile Chrome and enable your IP (e.g., http://192.168.1.2:5173).
Backend spawn UNKNOWN	Windows Node process execution path failure.	Solved via python -m uvicorn execution in scripts/dev-backend.js.
Room capacity overflow	Exceeding 10 calls per room.	Managed gracefully with a 1008 "Room Full" WebSocket close frame.
Code-switching (Singlish/Tanglish)	Speakers mix English words into daily speech.	Prompt-level hybrid language handling; planned fine-tuned translation prompt adapters.
API rate limits at scale	Parallel WebSocket connections per room (up to 20).	Dynamic session sleep / lazy-loading connection pools; enterprise-tier Gemini API plan for production.

9.1 Translation & Linguistic Nuances

Colloquial vs. formal script variance: spoken Sinhala and Tamil differ significantly from their written literary forms (e.g., informal verb endings, colloquial spoken forms). The model may transcribe speech into formal written grammar or struggle with regional slang and dialects (e.g., Jaffna vs. Indian Tamil).

Code-switching & hybrid dialects (Singlish/Tanglish): speakers frequently mix English words into daily speech. Rapid code-switching within a single sentence can cause transient delays or misinterpretation of English loanwords as part of the target language.

9.2 Latency & Streaming Performance

Network & geographical round-trip time: Gemini Live API endpoints run in regional data centers, so users on high-latency mobile networks (>150 ms ping) may see an initial delay of 800 ms–1.2 s before the first audio packet plays; playback buffering mitigates jitter afterward.

Activity detection silence window: the Gemini Live API's activity detector uses a 500 ms silence threshold to determine a completed turn. Pausing mid-sentence for longer than this may cause the model to prematurely treat the pause as turn-complete and translate only part of a sentence.

9.3 API Quotas & Concurrency Boundaries

Each connected client in Auto-Detect Mode opens 2 parallel WebSocket streams to Gemini Live API:

- 1 active call = 2 Gemini connections

- 5 active calls (10 users) = 20 concurrent Gemini connections

Free-tier or standard Gemini API keys enforce strict concurrent-connection limits (HTTP 429). Production deployment requires an enterprise-tier API plan with higher concurrency quotas.

9.4 Browser & Mobile Audio Hardware Constraints

Insecure-context microphone block: modern browsers restrict getUserMedia microphone access to secure contexts (https:// or http://localhost). Accessing the app over a LAN IP (e.g., http://192.168.1.5:5173) from a mobile device will block the microphone unless HTTPS is configured.

Acoustic background noise & mic sensitivity: loud ambient environments feed continuous non-speech audio into the PCM buffer, which can keep Gemini's activity detector active and delay translation turns.

9.5 Summary Matrix & Future Roadmap

Limitation Area	Root Cause	Current Mitigation	Planned Improvement
Code-switching	Mixed language tokens	Prompt instructions for hybrid language handling	Fine-tuned translation prompt adapters
API rate limits	Parallel WebSocket connections per room	Dynamic session sleep & lazy-loading connection pools	Server-side speech pre-classifier to run 1 session instead of 2
Mic HTTPS requirement	Browser security policy	Nginx reverse proxy with SSL (certbot / Let's Encrypt)	Automated Docker Compose SSL setup
Background noise	Unfiltered mic buffer	Client-side noise suppression (echoCancellation, noiseSuppression)	WebRTC DSP preprocessor

10. Testing Strategy

10.1 How to Run Tests

1. Backend Automated Test Suite (pytest)

Uses pytest alongside FastAPI's TestClient (built on httpx).

```
cd Tamil-and-Sinhala-Converter
.\.venv\Scripts\Activate.ps1  # Windows PowerShell
pip install pytest httpx

# Run all tests
pytest backend/tests -v
```

Expected output:

```
============================= test session starts ==============================
platform win32 -- Python 3.11.x, pytest-8.x.x
rootdir: d:\Tamil-and-Sinhala-Converter
collected 1 item

backend/tests/test_health.py::test_health_endpoint PASSED               [100%]

============================== 1 passed in 0.42s ===============================
```

2. Frontend Static Analysis & Type Checking (tsc)

```
cd frontend
npx tsc --noEmit
```

10.2 Concurrency Testing

Verified across 4 parallel browser tabs and multi-device Wi-Fi connections (http://192.168.1.2:5173).

10.3 Test Coverage Analysis

Component / Layer	Status	Covered	Not Covered
REST API (/api/v1/health)	Covered	HTTP 200 responses, JSON schema, service metadata, API key state	N/A
Unicode Language Detector	Unit testable	Script codepoint verification (Sinhala/Tamil ranges)	Singlish / Tanglish (English characters)
WebSocket Router (/ws/translate)	Partially covered	Handshake, parameter parsing	Live streaming audio round-trips over a live network

Room Connection Manager	Unit testable	Registration, room creation, disconnect removal, broadcasting	Network packet drop scenarios
Google Gemini Live API	Not automated	Verified via manual live audio testing and telemetry console	Mocked in CI/CD to avoid API key consumption
Frontend AudioWorklet / Web Audio	Manual testing	Audio capture, visualization, buffer flushing	Automated headless-browser microphone input

10.4 Why External Gemini Live WebSockets Are Mocked in CI/CD

- **API key security & cost —** exposing production keys in CI risks quota exhaustion and leakage.
- **Nondeterministic AI outputs —** generative audio varies across runs, making exact-match assertions unsuitable.
- **Hardware microphone dependencies —** headless browser test runners lack physical audio input without virtual audio cable setups.

10.5 Recommended WebSocket Integration Test Pattern

Standard pattern for testing room management and connection lifecycle using pytest-asyncio and FastAPI's websocket_connect:

backend/tests/test_websocket_rooms.py

```
import pytest
from fastapi.testclient import TestClient
from backend.main import app

client = TestClient(app)

def test_websocket_connection_room_assignment():
    """
    Verifies that clients connecting to /ws/translate-auto
    are properly accepted and tagged with the requested room ID.
    """
    with client.websocket_connect("/ws/translate-auto?voice=Aoede&room=test_room_101") as
websocket:
        data = websocket.receive_json()
        assert data["type"] == "status"
        assert "test_room_101" in data["payload"]["message"]
```

Appendix A — Key Source Code Listings

This appendix reproduces the core implementation files referenced throughout the architecture sections, for engineering review.

A.1 Connection Manager & 10-Call Room Capacity Pool

File: backend/websocket/connection_manager.py

```
from typing import Set, Dict
from fastapi import WebSocket

MAX_CALLS_PER_ROOM = 10  # Maximum 10 active call channels per room cluster

class ConnectionManager:
    def __init__(self):
        # Map of room_id -> Set of active WebSocket connections
        self.rooms: Dict[str, Set[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, room_id: str = "default"):
        """Accept connection and enforce maximum 10-call capacity per room."""
        await websocket.accept()
        if room_id not in self.rooms:
            self.rooms[room_id] = set()

        # Enforce maximum concurrent callers (10 calls = 20 max clients)
        if len(self.rooms[room_id]) >= MAX_CALLS_PER_ROOM * 2:
            await websocket.send_json({
                "type": "error",
                "payload": {"message": f"Room '{room_id}' reached maximum call capacity
({MAX_CALLS_PER_ROOM} calls)."}
            })
            await websocket.close(code=1008, reason="Room full")
            return False

        self.rooms[room_id].add(websocket)
        return True

    async def broadcast_bytes_except(self, data: bytes, exclude: WebSocket, room_id: str =
"default"):
        """Broadcast binary audio payload to all clients in the room except the speaker."""
        if room_id in self.rooms:
            for connection in self.rooms[room_id]:
                if connection != exclude:
                    try:
                        await connection.send_bytes(data)
                    except Exception:
                        pass
```

A.2 Trilingual Script Language Detector Engine

File: backend/websocket/auto_stream_handler.py

```
_CODE_LANG = {"si": "Sinhala", "ta": "Tamil", "en": "English"}
_OPPOSITE = {"ta": "si", "si": "ta", "en": "si"}

def _detect_language(text: str) -> str | None:
    """
    Detect whether text is predominantly Sinhala, Tamil, or English by inspecting
    Unicode script ranges.
    """
    si = ta = en = 0
    for ch in text:
        cp = ord(ch)
        if 0x0D80 <= cp <= 0x0DFF:      # Sinhala Unicode Range
            si += 1
        elif 0x0B80 <= cp <= 0x0BFF:    # Tamil Unicode Range
            ta += 1
        elif (0x0041 <= cp <= 0x005A) or (0x0061 <= cp <= 0x007A):  # English ASCII
            en += 1
    total = si + ta + en
    if total == 0:
        return None
    if si / total >= 0.5:
        return "Sinhala"
    if ta / total >= 0.5:
        return "Tamil"
    if en / total >= 0.5:
        return "English"
    return None
```

A.3 Multimodal Dual-Session Gemini Live Stream Runner

File: backend/websocket/auto_stream_handler.py

```
async def run_session(target_code: str, in_queue: asyncio.Queue):
    config = _make_config(target_code)
    while not client_disconnected.is_set():
        try:
            async with ai_client.aio.live.connect(model=model, config=config) as session:
                logger.info(f"Gemini Live Session [{target_code}] opened for room: {room_id}")

                # Task 1: Forward client PCM audio to Gemini
                async def forward_audio():
                    while not client_disconnected.is_set():
                        chunk = await asyncio.wait_for(in_queue.get(), timeout=0.3)
                        if chunk:
                            await session.send_realtime_input(
                                audio=types.Blob(data=chunk, mime_type="audio/pcm;rate=16000")
                            )

                # Task 2: Receive translated audio & text from Gemini
                async def receive_responses():
                    async for response in session.receive():
                        sc = response.server_content
                        if not sc: continue

                        # Detect language from input transcription
                        if sc.input_transcription and sc.input_transcription.text:
                            transcript = sc.input_transcription.text
                            detected = _detect_language(transcript)
                            if detected == source_lang and state["active"] != target_code:
                                state["active"] = target_code  # Switch active output session
```

```
                        # Only broadcast audio from the active translator channel
                        if state["active"] != target_code: continue

                        # Stream synthesized PCM audio bytes to room participants
                        if sc.model_turn:
                            for part in sc.model_turn.parts:
                                if part.inline_data and part.inline_data.data:
                                    await
manager.broadcast_bytes_except(part.inline_data.data, client_ws, room_id)

                await asyncio.gather(forward_audio(), receive_responses())
        except Exception as ex:
            await asyncio.sleep(1)
```

A.4 FastAPI Gateway WebSocket Route

File: backend/main.py

```
@app.websocket("/ws/translate-auto")
async def websocket_auto_translator_endpoint(
    websocket: WebSocket,
    voice: str = "Aoede",
    room: str = "default"
):
    """Bidirectional auto-detect endpoint accepting room ID and voice parameters."""
    connected = await manager.connect(websocket, room)
    if not connected:
        return  # Rejects connection if room reached 10-call capacity

    logger.info(f"Client connected (auto mode): {websocket.client}, room={room},
voice={voice}")
    try:
        await handle_auto_translation_stream(websocket, voice, room)
    except WebSocketDisconnect:
        logger.info(f"Client disconnected: {websocket.client}")
    finally:
        manager.disconnect(websocket, room)
```

A.5 Client WebSocket Connection Hook (React)

File: frontend/src/hooks/useAudioStream.ts

```
// Open the auto-detect WebSocket with dynamic room configuration
const connectAutoWebSocket = useCallback((voiceName: string) => {
  if (isWsConnectingRef.current) return;
  closeSocket();
  isWsConnectingRef.current = true;

  // Append room query parameter to backend WebSocket URL
  const wsUrl = `ws://${window.location.hostname}:8000/ws/translate-
auto?voice=${encodeURIComponent(voiceName)}&room=${encodeURIComponent(room)}`;
  addLog(`Connecting auto-detect mode (Room: ${room}) | voice: ${voiceName}`);

  const socket = new WebSocket(wsUrl);
  socketRef.current = socket;
  socket.binaryType = 'arraybuffer';
  nextPlaybackTimeRef.current = 0;

  socket.onopen = () => {
    setIsConnected(true);
    setIsRecording(true);
    setSessionState('AI_LISTENING');
  };
}, [addLog, closeSocket, playAudioChunk, room]);
```